Prepared for self-study · August 2026

Table of Contents

1. Why Design Patterns Matter at This Stage

- 1.1 What interviewers expect from 3–5 YOE candidates
- 1.2 How this guide is organized

2. SOLID Principles (the foundation)

- 2.1 Single Responsibility
- 2.2 Open/Closed
- 2.3 Liskov Substitution
- 2.4 Interface Segregation
- 2.5 Dependency Inversion

3. Creational Patterns

- 3.1 Singleton
- 3.2 Factory Method
- 3.3 Abstract Factory
- 3.4 Builder
- 3.5 Prototype

4. Structural Patterns

- 4.1 Adapter
- 4.2 Decorator
- 4.3 Facade
- 4.4 Proxy
- 4.5 Composite
- 4.6 Bridge
- 4.7 Flyweight

5. Behavioral Patterns

- 5.1 Strategy
- 5.2 Observer
- 5.3 Template Method
- 5.4 Command
- 5.5 Chain of Responsibility
- 5.6 State
- 5.7 Iterator
- 5.8 Mediator
- 5.9 Visitor
- 5.10 Memento

Design Patterns

A Practical Guide for Java Backend Developers

Target Audience: 3–5 Years Experience

Topic-wise, simple-language explanations of the classic Gang-of-Four patterns plus enterprise/Java patterns, with real Java code, Spring Framework connections, and interview-oriented tips.

6. Concurrency Patterns

- 6.1 Producer-Consumer
- 6.2 Thread Pool
- 6.3 Future / CompletableFuture
- 6.4 Immutable Object
- 6.5 Read-Write Lock

7. Enterprise & Java Backend Patterns

- 7.1 DAO (Data Access Object)
- 7.2 DTO (Data Transfer Object)
- 7.3 Repository
- 7.4 Service Layer
- 7.5 Dependency Injection / IoC
- 7.6 MVC
- 7.7 Unit of Work

8. Design Patterns Inside the Spring Framework

- 8.1 Where each GoF pattern shows up in Spring

9. Anti-Patterns to Avoid

- 9.1 Common misuse of patterns
- 9.2 Code smells that suggest a pattern is needed

10. How to Talk About Design Patterns in Interviews

- 10.1 The structured-answer framework
- 10.2 Pattern selection cheat sheet

11. Worked Mini Case Studies

- 11.1 Payment Gateway Integration (Strategy + Factory)
- 11.2 Notification System (Observer + Decorator)
- 11.3 Order Processing Pipeline (Chain of Responsibility + Command)

1. Why Design Patterns Matter at This Stage

1.1 What interviewers expect from 3–5 YOE candidates

At this level, knowing pattern *names* isn't enough — interviewers want to see that you reach for a pattern because it solves a real problem you've hit (rigid conditionals, tangled object creation, tight coupling between layers), not because it looks impressive. They expect you to:

- Recognize a code smell and name the pattern that fixes it.
- Explain the trade-off — every pattern adds a layer of indirection, which costs some simplicity.
- Connect patterns to the frameworks you use daily (Spring uses most of these patterns internally).
- Avoid over-engineering — knowing when *not* to apply a pattern is as important as knowing one.

> **Simple way to think about it:** A design pattern is not a piece of code you copy-paste. It's a reusable *shape* of a solution to a recurring problem — the same way "sort the smaller pile first" is a reusable shape of a solution, independent of whether you're sorting cards or files.

1.2 How this guide is organized

Each pattern below follows the same structure: the problem it solves in plain language, a short Java example, where you'd realistically use it in a backend/Spring codebase, and common pitfalls. Interview tip boxes highlight what to emphasize when the topic comes up in conversation.

2. SOLID Principles (the foundation)

Design patterns are concrete techniques; SOLID principles are the underlying design goals that most patterns exist to help you achieve. Know these cold — they come up constantly, patterns or not.

2.1 Single Responsibility Principle (SRP)

A class should have only one reason to change. A `OrderService` that also formats emails and writes logs to a custom file format has three reasons to change, not one — split it.

2.2 Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification. Instead of adding a new `if` branch to an existing method every time a new case appears, design so new behavior can be added via a new class implementing an interface (this is exactly what the Strategy pattern, Section 5.1, gives you).

2.3 Liskov Substitution Principle (LSP)

A subclass should be usable anywhere its parent class is expected, without breaking correctness. Classic violation: a `Square extends Rectangle` that overrides `setWidth()` to also change height — this breaks code that assumes setting width alone doesn't affect height.

2.4 Interface Segregation Principle (ISP)

Don't force a class to implement methods it doesn't need. Prefer several small, focused interfaces (`Readable` , `Writable`) over one fat interface (`Storage` with 15 methods).

2.5 Dependency Inversion Principle (DIP)

High-level modules shouldn't depend on low-level modules directly — both should depend on abstractions (interfaces). This is the principle that makes Dependency Injection (Section 7.5) possible and is the reason a `PaymentService` should depend on a `PaymentGateway` interface, not a concrete `StripeGateway` class.

> **Interview tip:** If asked "why do we use interfaces here?", the strongest answer ties back to DIP: it lets you swap implementations (real vs mock in tests, Stripe vs PayPal in prod) without touching the code that depends on them.

3. Creational Patterns

These patterns control *how objects are created*, so that object-creation logic doesn't leak all over the codebase.

3.1 Singleton

Creational Ensures one instance

Problem: Some objects (a config manager, a connection pool, a cache) should exist exactly once across the whole application. Creating multiple instances wastes resources or causes inconsistent state.

```
public class ConfigManager {
    private static final ConfigManager INSTANCE = new ConfigManager();
    private ConfigManager() {}
    public static ConfigManager getInstance() { return INSTANCE; }
}
```

Where you'd use it: Rarely written by hand in Spring apps — Spring beans are singleton by default (Section 8.1). Still worth knowing for non-Spring utility classes or when explaining how Spring's container achieves the same goal.

> **Pitfall:** Hand-rolled singletons make unit testing hard (can't easily mock a static instance) and can hide global mutable state, which causes subtle bugs under concurrency. Prefer letting a DI container manage the singleton lifecycle instead of writing your own.

3.2 Factory Method

Creational Delegates instantiation to subclasses

Problem: You need to create objects, but the exact class to instantiate depends on some condition, and you don't want that `if/else` chain scattered across the codebase.

```
public interface NotificationFactory {
    Notification create();
}
public class EmailNotificationFactory implements NotificationFactory {
    public Notification create() { return new EmailNotification(); }
}
public class SmsNotificationFactory implements NotificationFactory {
    public Notification create() { return new SmsNotification(); }
}
```

Where you'd use it: Any time you're creating one of several related implementations based on a type/enum — payment gateway selection, report generators, parser selection based on file type.

3.3 Abstract Factory

Creational | Factory of factories

Problem: You need to create *families* of related objects that must be used together (e.g. a UI toolkit's buttons and checkboxes must all match the same theme). A regular factory only handles one product type; an abstract factory handles a whole family.

```
public interface CloudResourceFactory {
    Storage createStorage();
    Compute createCompute();
}
public class AwsResourceFactory implements CloudResourceFactory {
    public Storage createStorage() { return new S3Storage(); }
    public Compute createCompute() { return new Ec2Compute(); }
}
```

Where you'd use it: Multi-cloud or multi-tenant backend systems where a whole set of related components must come from the same provider/tenant configuration consistently.

3.4 Builder

Creational | Step-by-step construction

Problem: A class has many optional constructor parameters, leading to "telescoping constructors" (multiple overloaded constructors) or huge constructor calls that are error-prone and hard to read.

```
Order order = Order.builder()
    .customerId(101)
    .item("Laptop")
    .quantity(1)
    .discountCode("SAVE10")
    .build();
```

Where you'd use it: Constantly — DTOs/entities with many optional fields. Lombok's `@Builder` annotation generates this pattern automatically in most Spring Boot codebases, so you rarely hand-write the boilerplate, but you should recognize and be able to explain what it's doing.

> **Java angle:** `StringBuilder` and Java's own `Stream.Builder` are built-in examples of this pattern you already use.

3.5 Prototype

Creational | Clone instead of construct

Problem: Creating a new object from scratch is expensive (heavy initialization, DB lookups), but you often need many near-identical copies with small variations.

```
public class ReportTemplate implements Cloneable {
    public ReportTemplate clone() {
        return (ReportTemplate) super.clone(); // then tweak fields as needed
    }
}
```

Where you'd use it: Less common in typical Spring backends; shows up in scenarios like cloning a complex configuration object as a starting point for a variant, or in game/simulation-style systems. Worth knowing conceptually more than using frequently.

> **Pitfall:** Java's `Object.clone()` does a shallow copy by default — nested mutable objects are shared between the original and the clone unless you deep-copy them explicitly. This is a classic interview follow-up question.

4. Structural Patterns

These patterns deal with how classes and objects are *composed* into larger structures, without making the whole system rigid or fragile.

4.1 Adapter

Structural Bridges incompatible interfaces

Problem: You need to use a class whose interface doesn't match what your code expects — commonly a third-party library or a legacy system.

```
public interface PaymentProcessor {
    void charge(BigDecimal amount);
}
public class LegacyPaymentAdapter implements PaymentProcessor {
    private final LegacyBillingSystem legacy;
    public void charge(BigDecimal amount) {
        legacy.billCustomer(amount.doubleValue()); // adapts the call
    }
}
```

Where you'd use it: Wrapping a third-party SDK behind your own interface so the rest of your codebase doesn't depend on that vendor's API shape directly (also makes swapping vendors later much easier).

4.2 Decorator

Structural Adds behavior dynamically

Problem: You want to add responsibilities to an individual object (logging, caching, compression) without modifying its class or creating an explosion of subclasses for every combination.

```
public interface DataSource {
    void write(String data);
}
public class EncryptedDataSource implements DataSource {
    private final DataSource wrapped;
    public EncryptedDataSource(DataSource wrapped) { this.wrapped = wrapped; }
    public void write(String data) { wrapped.write(encrypt(data)); }
}
```

Where you'd use it: Java I/O streams are the textbook example (`new BufferedReader(new InputStreamReader(in))`). In Spring, this pattern's spirit shows up in AOP and in wrapping beans (e.g. a caching decorator around a repository).

4.3 Facade

Structural Simplifies a complex subsystem

Problem: A subsystem has many moving parts (multiple classes, complex call order), and callers shouldn't need to know all of that detail just to perform a common task.

```
public class OrderFacade {
    public void placeOrder(OrderRequest req) {
        inventoryService.reserve(req);
        paymentService.charge(req);
        shippingService.schedule(req);
        notificationService.notifyCustomer(req);
    }
}
```

Where you'd use it: A Spring `@Service` class is very often acting as a facade over multiple repositories/other services — giving the controller layer one simple method to call.

4.4 Proxy

Structural Controls access to an object

Problem: You need to add a control layer in front of an object — lazy loading, access control, logging, caching — without the caller knowing it's not talking to the real object directly.

```
public class LazyImageProxy implements Image {
    private RealImage real;
    private final String path;
    public void display() {
        if (real == null) real = new RealImage(path); // loaded only when needed
        real.display();
    }
}
```

Where you'd use it: This is exactly how Spring AOP works — when you add `@Transactional` or `@Cacheable` to a method, Spring wraps your bean in a dynamic proxy that adds that behavior before/after delegating to your real method (Section 8.1).

> **Pitfall:** The classic "self-invocation" gotcha — calling an `@Transactional` method from another method *within the same class* bypasses the proxy entirely, so the annotation has no effect. A very common real-world Spring bug rooted directly in how the Proxy pattern works.

4.5 Composite

Structural Tree structures, uniform treatment

Problem: You need to treat individual objects and groups of objects the same way — e.g. a file and a folder (which contains files and other folders) should both respond to `getSize()`.

```
public interface FileSystemNode { long getSize(); }
public class File implements FileSystemNode { public long getSize() { return size; } }
public class Folder implements FileSystemNode {
    private List<FileSystemNode> children;
    public long getSize() { return children.stream().mapToLong(FileSystemNode::getSize).sum();
}
}
```

Where you'd use it: Hierarchical data — permission trees, category/subcategory structures, org charts, menu structures in a UI-serving API.

4.6 Bridge

Structural Decouples abstraction from implementation

Problem: You have two independent dimensions of variation (e.g. "type of notification" and "delivery channel") and don't want a combinatorial explosion of subclasses for every pairing.

```
public interface MessageSender { void send(String msg); } // implementation side
public abstract class Notification { // abstraction side
    protected MessageSender sender;
    abstract void notifyUser(String content);
}
```

Where you'd use it: Less common day-to-day, but useful to mention when a design has two truly independent axes of variation — it separates "what" from "how" so each can evolve on its own.

4.7 Flyweight

Structural Shares state to save memory

Problem: You need huge numbers of similar objects, and creating one full object per instance wastes memory. The pattern splits state into **intrinsic** (shared, immutable) and **extrinsic** (unique, passed in per use).

Where you'd use it: Rare in typical CRUD backends; more common in caching pools where many objects share common immutable configuration. Java's `Integer.valueOf()` caching small integers (-128 to 127) is a built-in real-world example.

5. Behavioral Patterns

These patterns deal with how objects *communicate and assign responsibility* to each other — the largest and most frequently used category in real backend code.

5.1 Strategy

Behavioral Interchangeable algorithms

Problem: You have multiple ways to perform an operation (different discount calculations, different payment gateways, different sorting rules) and want to switch between them at runtime without a giant `if/else` or `switch`.

```
public interface DiscountStrategy {
    BigDecimal apply(BigDecimal price);
}
public class SeasonalDiscount implements DiscountStrategy {
    public BigDecimal apply(BigDecimal price) { return price.multiply(BigDecimal.valueOf(0.9));
}
}
// usage
public class PriceCalculator {
    private DiscountStrategy strategy;
    public BigDecimal calculate(BigDecimal price) { return strategy.apply(price); }
}
```

Where you'd use it: Extremely common — payment gateway selection, pricing rules, validation rules, sorting/comparison logic. This is usually the very first pattern people reach for once they notice a growing `if/else` chain.

> **Java angle:** With Spring, you can inject `List<DiscountStrategy>` and let Spring autowire all implementations, then pick the right one by a key (e.g. a Map built from a `@Qualifier` or a custom enum lookup) — a very common, clean way to apply OCP (Section 2.2) in Spring code.

5.2 Observer

Behavioral One-to-many notification

Problem: When one object's state changes, several other objects need to react, but the subject shouldn't need to know the concrete details of its observers.

```
public interface OrderObserver { void onOrderPlaced(Order order); }
public class OrderSubject {
    private List<OrderObserver> observers = new ArrayList<>();
    public void notifyAll(Order order) {
        observers.forEach(o -> o.onOrderPlaced(order));
```

```
    }
}
```

Where you'd use it: Event-driven code within a single service. Spring's `ApplicationEventPublisher` + `@EventListener` is a ready-made implementation of this pattern — you rarely write the plumbing yourself in Spring Boot.

> **Simple analogy:** Like subscribing to a YouTube channel — the channel (subject) doesn't know who you are individually, it just notifies everyone on its subscriber list when it publishes something new.

5.3 Template Method

Behavioral | Fixed skeleton, variable steps

Problem: Several classes share the same overall algorithm structure, but a few steps differ. You want to avoid duplicating the shared steps in every class.

```
public abstract class ReportGenerator {
    public final void generate() { // template method - the skeleton
        fetchData();
        formatData();
        save();
    }
    protected abstract void fetchData();
    protected abstract void formatData();
    protected void save() { /* shared default implementation */ }
}
```

Where you'd use it: Import/export pipelines, multi-step validation flows with a common shape but different rules per type.

> **Java angle:** Spring's `JdbcTemplate`, `RestTemplate` / `WebClient`, and `TransactionTemplate` are all named after — and literally implement — this pattern: they handle the repetitive boilerplate (open connection, handle exceptions, close resources) and let you plug in just the variable part (your SQL, your request logic).

5.4 Command

Behavioral | Encapsulates a request as an object

Problem: You want to treat "an action to perform" as an object itself — so it can be queued, logged, undone, or executed later/elsewhere, decoupled from the caller that triggered it.

```
public interface Command { void execute(); }
public class CancelOrderCommand implements Command {
    private final Order order;
    public void execute() { order.cancel(); }
```

```
}
// A queue of Command objects can now be executed, retried, or logged uniformly.
```

Where you'd use it: Job/task queues, undo-able operations, encapsulating a unit of work before publishing it to a message queue (Kafka/RabbitMQ) for asynchronous execution.

5.5 Chain of Responsibility

Behavioral | Passes a request along a chain of handlers

Problem: A request needs to go through a series of independent processing steps, and not every request needs every step — you want to add/remove/reorder steps without touching a giant method.

```
public abstract class ValidationHandler {
    protected ValidationHandler next;
    public void setNext(ValidationHandler next) { this.next = next; }
    public void handle(Request req) {
        if (!isValid(req)) { reject(req); return; }
        if (next != null) next.handle(req);
    }
    protected abstract boolean isValid(Request req);
}
```

Where you'd use it: Almost every request-processing pipeline you've touched: Servlet `Filter` chains, Spring Security's `FilterChain`, and request validation/enrichment pipelines are all this pattern in action.

5.6 State

Behavioral | Behavior changes with internal state

Problem: An object's behavior needs to change based on its current state (an order that's NEW, PAID, SHIPPED, or CANCELLED behaves differently), and you want to avoid a sprawling `if (state == X)` chain scattered across many methods.

```
public interface OrderState {
    void ship(OrderContext ctx);
    void cancel(OrderContext ctx);
}
public class PaidState implements OrderState {
    public void ship(OrderContext ctx) { ctx.setState(new ShippedState()); }
    public void cancel(OrderContext ctx) { ctx.setState(new CancelledState()); }
}
```

Where you'd use it: Order/ticket/workflow status transitions — a very natural fit for any entity with a well-defined state machine (order lifecycle, approval workflows, subscription status).

5.7 Iterator

Behavioral | Sequential access without exposing internals

Problem: You want to traverse a collection's elements without exposing how the collection is internally structured (array, linked list, tree).

Where you'd use it: You already use this constantly — every Java `Iterator`, enhanced for-loop, and `Stream` pipeline is this pattern. Rarely implemented from scratch; important mainly to recognize and name correctly.

5.8 Mediator

Behavioral | Centralizes complex communication

Problem: Many objects need to talk to each other, and letting them reference each other directly creates a tangled web of dependencies. A mediator centralizes that communication.

Where you'd use it: An orchestrator service in a Saga pattern (recall the System Design guide, Section 10.3) is essentially a Mediator — it tells each participating service what to do next instead of having them call each other directly.

5.9 Visitor

Behavioral | Adds operations without changing classes

Problem: You need to add a new operation across a family of classes (e.g. export-to-XML, calculate-tax) without modifying each class every time a new operation is needed.

Where you'd use it: Compilers, AST processing, and some serialization frameworks use this heavily. It's less common in typical CRUD backend code — useful mainly to recognize when a codebase has a stable set of classes but a frequently growing set of operations on them.

5.10 Memento

Behavioral | Capture and restore state

Problem: You need to save an object's internal state so it can be restored later (undo functionality), without violating encapsulation by exposing that state directly.

Where you'd use it: Undo/redo features, saving draft versions of a document/form, or snapshotting state before a risky operation so it can be rolled back.

6. Concurrency Patterns

Backend services are inherently concurrent (multiple requests at once), so these patterns come up constantly in practice, even if they're less often framed explicitly as "patterns."

6.1 Producer-Consumer

Decouples work generation from work processing using a shared queue, so producers and consumers can run at their own pace. Java's `BlockingQueue` implementations (`ArrayBlockingQueue`, `LinkedBlockingQueue`) implement this directly. At a larger scale, this is exactly what a message queue like Kafka/RabbitMQ does between services.

6.2 Thread Pool

Reuses a fixed set of worker threads instead of creating a new thread per task — avoids the overhead and resource exhaustion of unbounded thread creation. Java's `ExecutorService` (`Executors.newFixedThreadPool()`, or better, a manually configured `ThreadPoolExecutor` for production) is the standard implementation. Tomcat's request-handling thread pool underlies every Spring MVC application.

6.3 Future / CompletableFuture

Represents the result of an asynchronous computation that isn't ready yet, letting you compose async operations without blocking threads unnecessarily.

```
CompletableFuture<Order> orderFuture = orderService.fetchAsync(orderId);
CompletableFuture<Void> result = orderFuture
    .thenApply(this::enrichWithPricing)
    .thenAccept(this::sendConfirmation);
```

Where you'd use it: Calling multiple independent downstream services in parallel and combining their results, without dedicating one blocking thread per call.

6.4 Immutable Object

An object whose state can't change after construction (all fields `final`, no setters). Immutable objects are automatically thread-safe — no locking needed, because there's no mutable shared state to protect. Java records (Java 16+) are purpose-built for this.

6.5 Read-Write Lock

Allows multiple concurrent readers but only one exclusive writer, improving throughput for read-heavy shared data compared to a single exclusive lock. Java's `ReentrantReadWriteLock` implements

this directly — useful for an in-memory cache/config object that's read constantly but updated rarely.

7. Enterprise & Java Backend Patterns

Beyond the classic GoF catalogue, these are architectural patterns you'll use in nearly every Spring Boot service, often without thinking of them as "patterns" at all.

7.1 DAO (Data Access Object)

Encapsulates all database access logic behind an interface, so the rest of the application doesn't know or care whether data comes from MySQL, MongoDB, or an in-memory store. Separates persistence concerns from business logic.

7.2 DTO (Data Transfer Object)

A plain object used to carry data across layers/boundaries (e.g. from your service to an API response), deliberately decoupled from your JPA entity. Prevents leaking internal database structure (and lazy-loading issues) directly into your API contract.

> **Pitfall:** Returning JPA entities directly from a REST controller is a very common real-world mistake — it couples your API contract to your DB schema and can trigger `LazyInitializationException` during JSON serialization. Map to a DTO at the service/controller boundary instead.

7.3 Repository

Similar in spirit to DAO but framed at a higher level of abstraction — presents a collection-like interface over your domain objects (`findById`, `save`, `findByStatus`) rather than exposing raw persistence operations. Spring Data JPA's `JpaRepository` is the ready-made implementation you use daily.

7.4 Service Layer

Holds business logic and orchestrates calls across one or more repositories/other services, keeping controllers thin (just handling HTTP concerns) and repositories dumb (just handling persistence). The `@Service`-annotated class in nearly every Spring Boot app.

7.5 Dependency Injection / Inversion of Control (IoC)

Instead of a class creating its own dependencies (`new StripeGateway()`), the dependencies are "injected" from outside (usually via constructor), and the class just declares what it needs via an interface. This is DIP (Section 2.5) put into practice. Spring's `ApplicationContext` is an IoC container that wires all your `@Component` / `@Service` / `@Repository` beans together automatically.

> **Java angle:** Prefer constructor injection over field injection (`@Autowired` on a field) — it makes dependencies explicit, immutable (`final` fields), and far easier to unit test without spinning up a Spring context.

7.6 MVC (Model-View-Controller)

Separates an application into Model (data/business logic), View (presentation), and Controller (handles input, coordinates between the two). In a typical Spring Boot REST API, the "view" is really just the JSON response body, but the separation of Controller (HTTP handling) from Service (business logic) from Model/Entity (data) is the same underlying idea.

7.7 Unit of Work

Tracks a set of changes during a business transaction and commits them all together at the end, ensuring consistency. This is essentially what JPA's `EntityManager` /Hibernate `Session` does under the hood — it tracks "dirty" entities within a transaction and flushes all changes together when the transaction commits (or rolls all of them back together on failure).

8. Design Patterns Inside the Spring Framework

One of the strongest signals you can give in an interview is connecting a pattern to something you've actually used. Spring is built almost entirely out of the patterns above.

8.1 Where each GoF pattern shows up in Spring

Pattern	Where it appears in Spring
Singleton	Default bean scope — one shared instance per Spring container by default
Factory Method	`BeanFactory` / `ApplicationContext.getBean()` creates beans without the caller using `new`
Proxy	AOP — `@Transactional`, `@Cacheable`, `@Async` all work via dynamic proxies wrapping your bean
Template Method	`JdbcTemplate`, `RestTemplate`, `TransactionTemplate`
Observer	`ApplicationEventPublisher` + `@EventListener`
Chain of Responsibility	Servlet `Filter` chain; Spring Security's security filter chain
Decorator	Bean post-processors wrapping beans; `HandlerInterceptor` chains around controller calls
Strategy	`PlatformTransactionManager` implementations, `PasswordEncoder` implementations
Builder	`UriComponentsBuilder`, `ResponseEntity.ok().body()` fluent APIs
Adapter	`HandlerAdapter` in Spring MVC's `DispatcherServlet` flow

> **Interview tip:** This table alone is one of the strongest things you can bring up unprompted — it shows you understand patterns are not academic, they're literally how the framework you use every day is built.

9. Anti-Patterns to Avoid

9.1 Common misuse of patterns

- **Pattern-itis / over-engineering** — wrapping a simple two-line method in a Strategy + Factory + Builder because "patterns are good practice." Apply a pattern only when the problem it solves actually exists in your code.
- **God object** — a single class doing everything (violates SRP), often the reason a codebase feels impossible to change safely.
- **Anemic domain model** — entities that are just data bags (getters/setters only) with all logic dumped into service classes; not always wrong, but worth recognizing as a deliberate trade-off, not an accident.
- **Singleton abuse** — using a hand-rolled singleton as a dumping ground for global mutable state, making code hard to test and reason about under concurrency.

9.2 Code smells that suggest a pattern is needed

Code smell	Likely pattern fix
Long chain of `if/else` or `switch` picking behavior	Strategy
Constructors with many optional parameters	Builder
Class creation logic duplicated across the codebase	Factory Method
Need to add cross-cutting behavior (logging, caching) without touching core logic	Decorator / Proxy (AOP)
Tightly coupled classes calling each other directly in a web	Mediator
Multi-step process where not every request needs every step	Chain of Responsibility
Object behaves differently depending on a status field checked everywhere	State

10. How to Talk About Design Patterns in Interviews

10.1 The structured-answer framework

1. **Name the problem, not the pattern** — start with "we had a growing if/else picking the discount logic," not "I used Strategy."
2. **Explain the shape of the fix** — describe the interface + implementations, in plain language before jumping to code.
3. **Name the pattern** — now say "this is the Strategy pattern" to show you know the vocabulary, not just the mechanics.
4. **State the trade-off** — more classes/indirection in exchange for easier extension and testability. Showing you know the cost, not just the benefit, is what separates a mid-level answer from a junior one.
5. **Tie it to something real** — mention where the same pattern shows up in Spring (Section 8.1) if relevant; it signals depth beyond memorized definitions.

10.2 Pattern selection cheat sheet

You need to...	Reach for
Swap an algorithm/behavior at runtime	Strategy
React to an event happening elsewhere	Observer
Reuse a fixed process with a few varying steps	Template Method
Add behavior without touching the original class	Decorator or Proxy
Simplify a complex subsystem for callers	Facade
Make an incompatible API fit your code	Adapter
Build a complex object step by step	Builder
Pick an implementation based on a type/condition	Factory Method
Process a request through independent, optional steps	Chain of Responsibility
Model an entity whose behavior depends on its status	State

11. Worked Mini Case Studies

11.1 Payment Gateway Integration (Strategy + Factory)

Problem

Support multiple payment providers (Stripe, PayPal, Razorpay) selected at runtime based on the customer's region/preference, without a growing `if/else` in the checkout flow.

Design

- Define a `PaymentGateway` interface with `charge()` / `refund()` (Strategy, Section 5.1).
- Implement one class per provider; each wraps that provider's SDK behind the common interface (also acting as an Adapter, Section 4.1, if the SDK's shape doesn't match your interface directly).
- A `PaymentGatewayFactory` (Section 3.2) picks the right implementation based on a `PaymentMethod` enum, so `CheckoutService` never references a concrete gateway class.

Why this shape

Adding a fourth provider later means adding one new class and one factory case — no existing code is touched (Open/Closed Principle, Section 2.2).

11.2 Notification System (Observer + Decorator)

Problem

When an order's status changes, multiple independent things should happen — send an email, update an analytics dashboard, notify a warehouse system — without the order code knowing about all of them directly.

Design

- `OrderService` publishes an `OrderStatusChangedEvent` (Observer, Section 5.2) using Spring's `ApplicationEventPublisher` rather than calling each interested system directly.
- Each interested system (`EmailNotifier` , `AnalyticsListener` , `WarehouseNotifier`) listens independently via `@EventListener` — adding a new listener never requires touching `OrderService` .
- The email-sending logic itself is wrapped with a `RetryDecorator` and a `LoggingDecorator` (Decorator, Section 4.2) so retry/logging behavior can be composed without modifying the core email-sending class.

11.3 Order Processing Pipeline (Chain of Responsibility + Command)

Problem

An incoming order must pass through several independent checks — fraud check, inventory check, discount validation — where the exact set of checks varies by order type, and each check should be

able to short- circuit the pipeline.

Design

- Each check is a `OrderHandler` in a Chain of Responsibility (Section 5.5); handlers are assembled into different chains per order type (e.g. digital orders skip the inventory check).
- Once validated, the actual processing step (charge payment, reserve stock, schedule shipment) is wrapped as a `Command` object (Section 5.4) and pushed onto a queue for asynchronous execution, decoupling "the order was validated" from "the order was fully processed."

Why this shape

New validation rules are added as new handler classes without touching existing ones (OCP again); the Command objects give you a natural way to log, retry, or replay individual processing steps if a downstream step fails.

End of guide · Focus on recognizing the problem shape first — the pattern name is secondary to knowing why it fits.